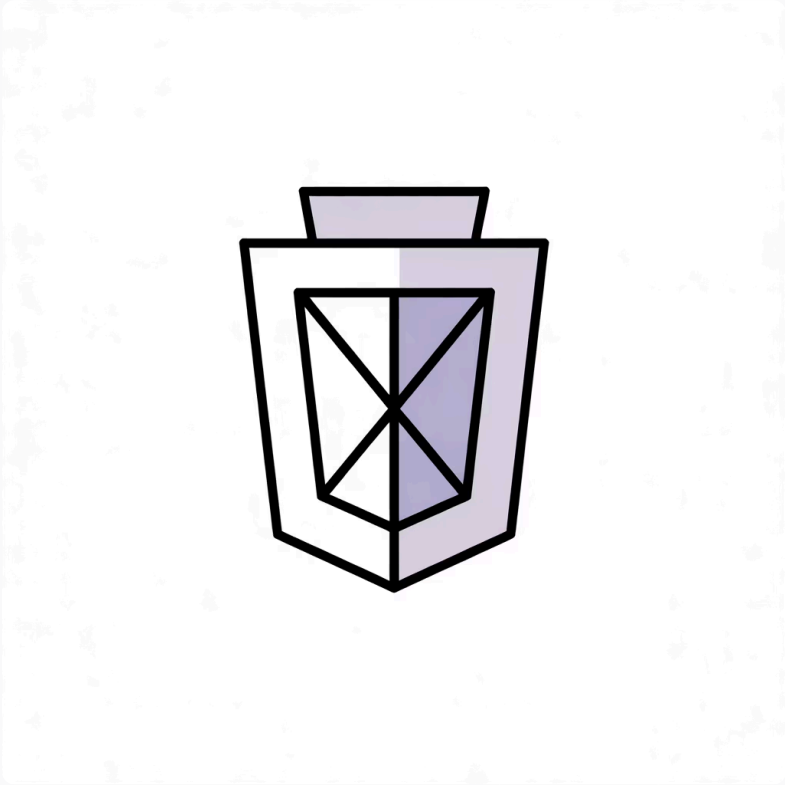
JavaScript - Jour 7

Programmation Orientée Objet



Objectifs pédagogiques de la journée

- Classes, constructeurs et méthodes
- Encapsulation, getters/setters et propriétés privées
- Héritage et polymorphisme
- Challenge de fin de journée

Rappels POO

La Programmation Orientée Objet (POO) est un paradigme qui structure le code autour d'objets, regroupant données (propriétés) et comportements (méthodes).

Données (Propriétés)

Les attributs et caractéristiques qui définissent l'état interne d'un objet.

Comportements (Méthodes)

Les fonctions ou actions que l'objet peut effectuer ou sur lesquelles il peut agir.

Les 4 piliers de la POO

Encapsulation

Regrouper les données et les méthodes qui les manipulent dans une seule unité (classe), protégeant l'état interne de l'objet.

Abstraction

Cacher les détails complexes d'implémentation et exposer seulement ce qui est pertinent à l'utilisateur.

Héritage

Permettre à une classe (enfant) d'acquérir les propriétés et comportements d'une autre classe (parent), favorisant la réutilisation du code.

Polymorphisme

Utiliser une interface commune pour différents types d'objets, permettant à une même méthode d'avoir des comportements différents selon l'objet qui l'appelle.

Avantages de la POO

Réutilisabilité

Les classes et les composants peuvent être réutilisés dans différentes parties de l'application ou dans de nouveaux projets, réduisant ainsi le temps de développement et les erreurs.

Maintenabilité

Grâce à la structure modulaire et à l'encapsulation, le code est plus facile à comprendre, à déboguer et à modifier sans affecter d'autres parties du système.

Modularité

Chaque classe a une responsabilité bien définie, ce qui favorise la décomposition des problèmes complexes en modules plus petits et gérables.

Scalabilité

La POO facilite l'ajout de nouvelles fonctionnalités et l'extension des applications existantes, permettant à un système de croître sans devenir excessivement complexe.

PROJET FIL ROUGE

Système de gestion de guilde RPG

Tout au long de cette journée, vous allez créer un système complet de gestion de guilde pour un jeu de rôle, en partant de bases simples pour y ajouter progressivement des fonctionnalités avancées.

Contexte : Vous êtes le maître de guilde et votre mission est de gérer vos aventuriers, leurs quêtes, leur inventaire et leurs combats !

Maître de Guilde

Votre rôle central dans l'aventure.

Aventuriers

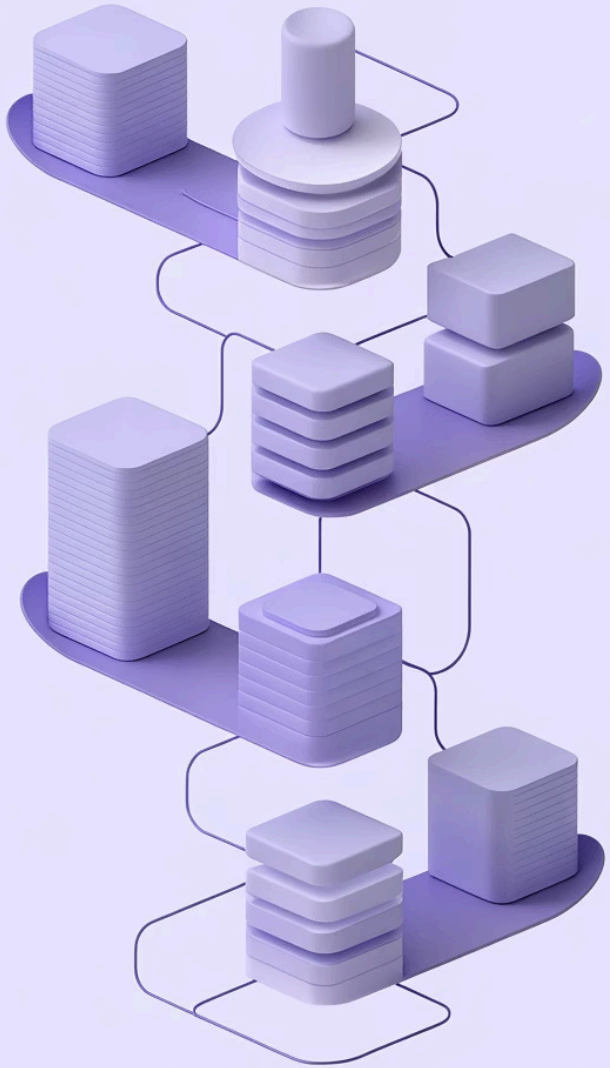
Gérez vos personnages et leurs compétences.

Quêtes & Missions

Organisez les expéditions de votre guilde.

Inventaire & Équipement

Suivez les ressources et l'équipement de votre guilde.



MODULE

Classes, constructeurs et méthodes

De l'objet littéral à la classe

Objet littéral (ce que vous connaissez déjà)

```
const warrior = {
  name: "Thorin",
  strength: 15,

  attack() {
    console.log(`${warrior.name} attaque avec ${warrior.strength} de force !`);
  }
};

const mage = {
  name: "Gandalf",
  strength: 5,

  attack() {
    console.log(`${mage.name} attaque avec ${mage.strength} de force !`);
  }
};
```

Problème : Beaucoup de duplication ! Si vous voulez créer 50 personnages, vous devez copier-coller ce code 50 fois. Cela rend le code difficile à maintenir et sujet aux erreurs.

Solution : Les classes

Une classe est un modèle (blueprint) pour créer des objets avec les mêmes propriétés et méthodes.

```
class MaClasse {
  constructor(parametres) {
    this.proprietes = parametres;
  }

  methode() {
    // ...
  }
}

const instance = new MaClasse("argument");
```

Le mot-clé `this`

Dans une classe, `this` **fait référence à l'instance actuelle de l'objet.**

Le constructeur

Le constructeur est une méthode spéciale au sein d'une classe. Il est appelé automatiquement à chaque fois que vous créez une nouvelle instance (un nouvel objet) de cette classe, en utilisant le mot-clé `new`.

```
class Person {
  constructor(name) {
    this.name = name;
  }
}

const person = new Person("Gauthier");
```

Le constructeur permet d'initialiser l'état initial de chaque objet créé, garantissant que toutes les propriétés essentielles sont définies dès le départ.

Rappel de la terminologie POO

Classe vs Instance

Une **classe** est un modèle abstrait (un plan) pour créer des objets. Une **instance** est un objet concret créé à partir de ce modèle, avec ses propres valeurs.

Variable vs Propriété

Une **variable** stocke une donnée dans un programme. Une **propriété** est une variable spécifique à une classe ou une instance, décrivant son état.

Fonction vs Méthode

Une **fonction** est un bloc de code exécutable. Une **méthode** est une fonction associée à une classe ou une instance, définissant son comportement.

⚠ Il est important de nommer les classes en PascalCase !

MISE EN PRATIQUE

Votre challenge

CONSIGNE PRINCIPALE

Créez la classe Character

À partir des exemples et explications du module actuel, créez une classe `Character` permettant d'instancier plusieurs héros pour notre guilde RPG, chacun avec ses propriétés.

BONUS 1

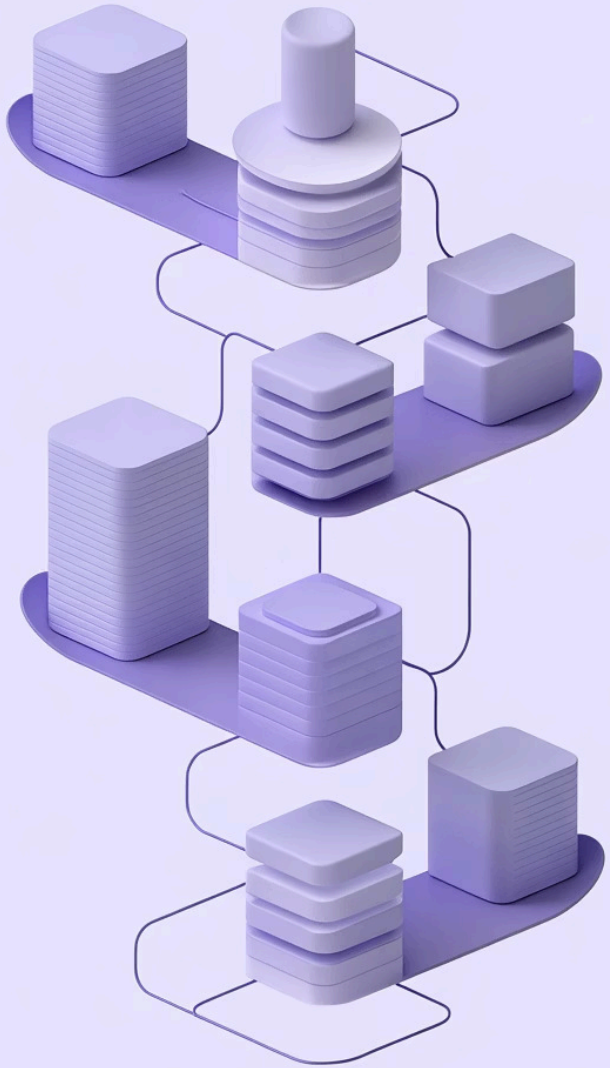
Intégrez les données json-server

Utilisez l'API `json-server` (vue vendredi) pour récupérer et initialiser les informations de vos personnages.

BONUS 2

Ajoutez une méthode attack()

Implémentez une méthode `attack()` qui affiche un `console.log` en utilisant les propriétés (nom et force) de l'instance pour simuler une attaque.



MODULE

Encapsulation, getters/setters et propriétés privées

L'encapsulation : pourquoi protéger les données ?

Actuellement, n'importe qui peut modifier directement les propriétés d'un objet, ce qui peut entraîner des incohérences et des comportements inattendus.

```
const legolas = new Character("Legolas", 10);

// Problème : on peut tricher !
legolas.strength = 999999;    // Force irréaliste !
legolas.level = 100;          // Progression non méritée !
legolas.hp = -50;             // Point de vie illogique !
```

⚠ **Le problème :** Une modification directe des propriétés rend l'état de nos objets vulnérable à la corruption. Cela peut permettre des tricheries, introduire des données illogiques ou casser la logique interne de notre application.

MODULE

L'encapsulation : les principes clés

Propriétés privées

Masquer l'implémentation interne d'un objet en rendant ses propriétés non directement accessibles depuis l'extérieur.

Accès contrôlé (Getters & Setters)

Fournir des méthodes publiques spéciales, appelées "getters" (pour lire les données) et "setters" (pour les modifier), afin de contrôler et de valider les interactions avec les propriétés internes.

Exemple : Encapsulation d'un compte bancaire

```
class CompteEnBanque {
  #solde;  // Propriété privée

  constructor(soldeInitial) {
    this.#solde = soldeInitial;
  }

  deposer(montant) {
    if (montant > 0) {
      this.#solde += montant;
      console.log(`Dépôt de ${montant}€. Nouveau solde : ${this.#solde}€`);
    } else {
      console.log("Le montant doit être positif !");
    }
  }

  retirer(montant) {
    if (montant > 0 && montant <= this.#solde) {
      this.#solde -= montant;
      console.log(`Retrait de ${montant}€. Nouveau solde : ${this.#solde}€`);
    } else {
      console.log("Montant invalide ou solde insuffisant !");
    }
  }

  afficherSolde() {
    console.log(`Solde actuel : ${this.#solde}€`);
  }
}

const compte = new CompteEnBanque(1000);
compte.deposer(500);
compte.retirer(200);
compte.retirer(2000); // Erreur attendue (solde insuffisant)

// Accès direct impossible :
console.log(compte.#solde);
compte.#solde = 999999;
```

Getters et Setters : Deux approches

Pour contrôler l'accès et la modification des propriétés de nos objets, JavaScript nous offre deux syntaxes principales pour implémenter des getters et des setters.

1. Syntaxe Classique (méthodes nommées)

Les getters et setters peuvent être définis comme des méthodes ordinaires au sein de votre classe, avec des noms descriptifs.

2. Syntaxe JavaScript (mots-clés `get` et `set`)

JavaScript propose des mots-clés dédiés qui permettent une syntaxe plus élégante pour les getters et les setters.

Première approche : Syntaxe Classique

```
class MyClass {
  #privateProperty; // Propriété privée

  constructor(initialValue) {
    this.setPrivateProperty(initialValue); // Utilise le setter pour l'initialisation
  }

  // Méthode dédiée à la modification (Setter)
  setPrivateProperty(newValue) {
    this.#privateProperty = newValue;
  }

  // Méthode dédiée à l'accès (Getter)
  getPrivateProperty() {
    return this.#privateProperty;
  }
}

// Utilisation
const instance = new MyClass(100);
instance.setPrivateProperty(250);     // Valide
console.log(instance.getPrivateProperty()); // Affiche 250
```

Seconde approche : Mots clés get et set

```
class MyClass {
  #privateProperty; // Propriété privée

  constructor(initialValue) {
    this.setPrivateProperty(initialValue); // Utilise le setter pour l'initialisation
  }

  // Propriété dédiée à la modification (Setter)
  set privateProperty(newValue) {
    this.#privateProperty = newValue;
  }

  // Propriété dédiée à l'accès (Getter)
  get privateProperty() {
    return this.#privateProperty;
  }
}

// Utilisation
const instance = new MyClass(100);
instance.privateProperty = 250;       // Valide
console.log(instance.privateProperty); // Affiche 250
```

MISE EN PRATIQUE

Refactorisation : Classe Character

1 Encapsulation complète

Transformez toutes les propriétés de la classe `Character` en propriétés privées (ex: `#hp`, `#mana`). Implémentez ensuite des getters et setters publics pour chacune d'elles, offrant un accès contrôlé et sécurisé.

2 Validation des données

Dans vos setters, ajoutez une logique de validation robuste. Par exemple, assurez-vous que les points de vie et le mana ne sont jamais négatifs, que la force est un nombre positif, et que les types de données sont corrects.

3 Getters calculés

Implémentez les getters suivants pour enrichir les informations de vos personnages :

- `get isAlive` : Retourne `true` si les points de vie sont supérieurs à 0.
- `get isExhausted` : Retourne `true` si le mana est égal à 0.
- `get hpPercentage` : Retourne le pourcentage de points de vie restants.
- `get manaPercentage` : Retourne le pourcentage de mana restant.

Ajoutez toutes les propriétés manquantes, à partir des données transmises via l'API !

MODULE

Des objets pour l'inventaire

Pour enrichir l'expérience de jeu, nous allons introduire une nouvelle classe `Item`. Celle-ci nous permettra de créer divers objets (armes, armures, potions, matériaux, objets de quête) qui pourront être ajoutés à l'inventaire privé de nos personnages.

```
class Item {
  constructor(name, type, value, description) {
    this.#name = name;
    this.#type = type;  // "arme", "armure", "potion", "materiau", "quete"
    this.#value = value;  // valeur en or
    this.#description = description;
  }

  displayInfos() {
    console.log(`[${this.#type}] ${this.#name} - ${this.#value} or`);
    console.log(`  ${this.#description}`);
  }
}
```

Cette classe simple définit les attributs dans le jeu et inclut une méthode pour afficher ses détails :

- `name` : Le nom de l'objet (ex: "Épée longue").
- `type` : La catégorie de l'objet ("arme", "armure", "potion", etc.).
- `value` : La valeur de l'objet en pièces d'or.
- `description` : Une courte description de l'objet.
- `displayInfos()` : Une méthode pour afficher les détails de l'objet dans la console.

MISE EN PRATIQUE

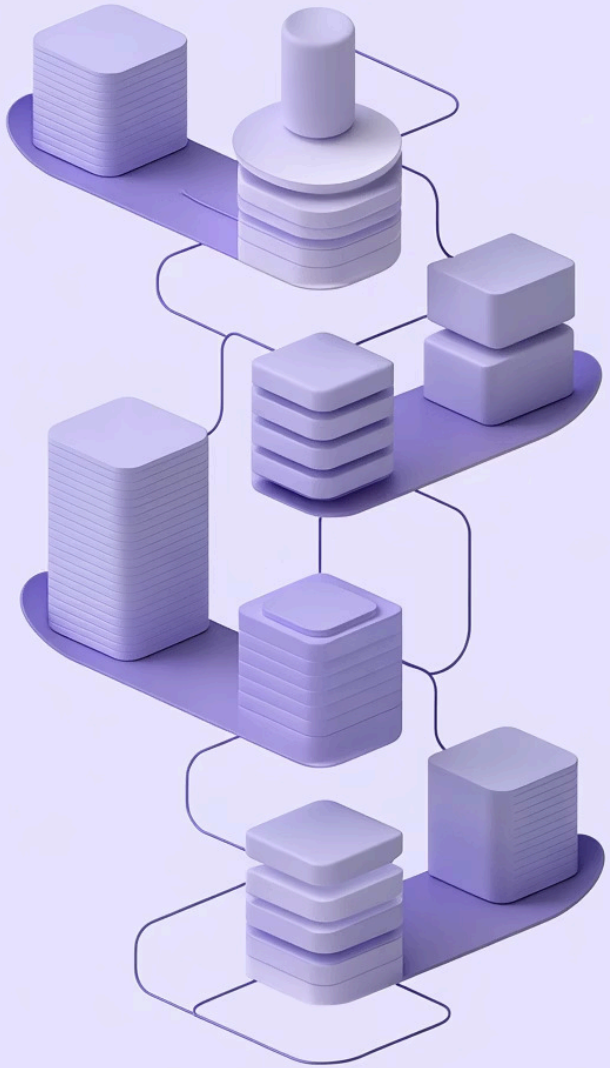
Intégration de l'inventaire

Nous allons maintenant enrichir notre classe `Character` en lui ajoutant un système d'inventaire, en réutilisant les concepts d'encapsulation et de méthodes.

1	2	3
Gestion des objets Ajoutez à la classe `Character` une propriété privée pour stocker les objets (par exemple, un tableau ou une `Map`).	**Méthodes d'interaction** Implémentez des méthodes publiques comme `addItem(item)` et `removeItem(item)` pour manipuler l'inventaire de manière contrôlée.	**Affichage de l'inventaire** Créez une méthode `displayInventory()` qui parcourt et liste les détails de tous les objets possédés par le personnage.

Bonus 1 : Pour une meilleure organisation et encapsulation, créez une classe dédiée `Inventory`.
Le personnage aurait alors une instance de cette classe, déléguant la gestion de son inventaire à cet objet.

Bonus 2 : Et pourquoi pas un système optimisé pour gérer les quantités d'un même objet, sans devoir le dupliquer dans l'inventaire ? 😏



MODULE

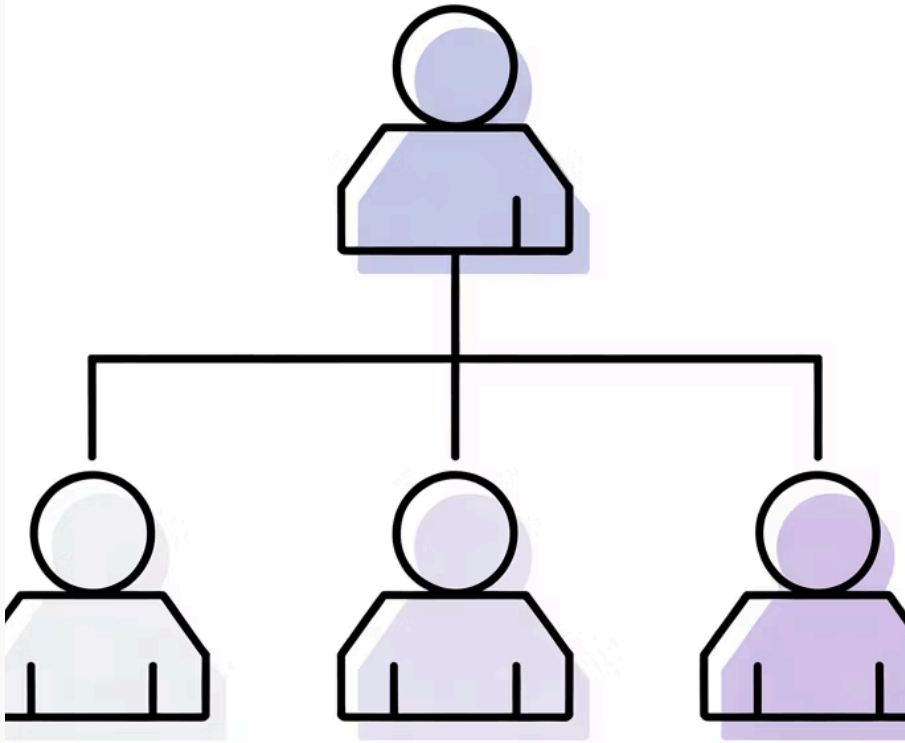
Héritage et polymorphisme

MODULE

L'héritage : créer des classes spécialisées

L'héritage est un concept fondamental en programmation orientée objet qui permet de définir une nouvelle classe (souvent appelée *classe enfant* ou *sous-classe*) à partir d'une classe existante (la *classe parente* ou *super-classe*).

La classe enfant hérite ainsi automatiquement de toutes les propriétés et méthodes de son parent, ce qui favorise la réutilisation de code. Elle peut ensuite ajouter ses propres caractéristiques uniques ou redéfinir le comportement de celles héritées, permettant de créer des versions plus spécialisées et adaptées à des besoins spécifiques.



SYNTAXE

L'héritage : Syntaxe de base

```
class ClasseParente {
  constructor(propriete1) {
    this.propriete1 = propriete1;
  }

  methodeParente() {
    console.log("Méthode de la classe parente");
  }
}

class ClasseEnfant extends ClasseParente {
  constructor(propriete1, propriete2) {
    super(propriete1);  // Appelle le constructeur du parent
    this.propriete2 = propriete2;
  }

  methodeEnfant() {
    console.log("Méthode spécifique à l'enfant");
  }
}

const enfant = new ClasseEnfant("valeur1", "valeur2");
enfant.methodeParente();  // Héritée du parent
enfant.methodeEnfant();   // Propre à l'enfant
```

EXEMPLE CONCRET

Héritage en action : Véhicules, Voitures et Motos

```
// Classe de base
class Vehicule {
  constructor(marque, modele, vitesseMax) {
    this.marque = marque;
    this.modele = modele;
    this.vitesseMax = vitesseMax;
    this.vitesseActuelle = 0;
  }

  accelerer(vitesse) {
    this.vitesseActuelle = Math.min(this.vitesseActuelle + vitesse, this.vitesseMax);
    console.log(`${this.marque} ${this.modele} accélère à ${this.vitesseActuelle} km/h`);
  }

  freiner() {
    this.vitesseActuelle = 0;
    console.log(`${this.marque} ${this.modele} s'arrête`);
  }
}

// Classe spécialisée : Voiture
class Voiture extends Vehicule {
  constructor(marque, modele, vitesseMax, nombrePortes) {
    super(marque, modele, vitesseMax);  // Appelle le constructeur du parent
    this.nombrePortes = nombrePortes;
  }

  klaxonner() {
    console.log("Tuut tuut !");
  }
}

// Classe spécialisée : Moto
class Moto extends Vehicule {
  constructor(marque, modele, vitesseMax, typeMoto) {
    super(marque, modele, vitesseMax);
    this.typeMoto = typeMoto;  // sportive, cross, routière
  }

  faireRoue() {
    console.log("Vroooom ! Roue arrière !");
  }
}

// Utilisation
const voiture = new Voiture("Peugeot", "308", 200, 5);
const moto = new Moto("Yamaha", "R1", 300, "sportive");

voiture.accelerer(100);  // Méthode héritée
voiture.klaxonner();     // Méthode propre

moto.accelerer(150);     // Méthode héritée
moto.faireRoue();        // Méthode propre
```

`Vehicule` (Classe parente)

Définit les propriétés et méthodes communes à tous les véhicules (marque, modèle, vitesse, accélérer, freiner).

`Voiture` (Classe enfant)

Hérite de `Vehicule` et ajoute des spécificités comme le nombre de portes et la méthode `klaxonner()`.

`Moto` (Classe enfant)

Hérite également de `Vehicule` et introduit son propre type et sa méthode unique `faireRoue()`.

SYNTAXE

Le mot-clé super

Le mot-clé `super` est un élément crucial en programmation orientée objet en JavaScript. Il permet aux classes enfants d'interagir directement avec leur classe parente, assurant ainsi une bonne gestion de l'héritage et une réutilisation efficace du code.

`super()` dans le constructeur

Lorsqu'utilisé dans le constructeur d'une classe enfant, `super()` appelle le constructeur de la classe parente.

C'est la première chose à faire dans le constructeur de l'enfant pour initialiser les propriétés héritées avant d'ajouter les propriétés spécifiques à l'enfant.

`super.méthode()` dans les méthodes

Utilisé à l'intérieur d'une méthode de la classe enfant, `super.méthode()` permet d'invoquer une méthode du même nom définie dans la classe parente.

C'est idéal pour étendre ou modifier le comportement d'une méthode parente sans la réécrire entièrement, en ajoutant des fonctionnalités avant ou après l'exécution de la méthode parente.

EXEMPLE CONCRET

Exemple d'utilisation de super

Ce code illustre comment `super` est utilisé pour appeler le constructeur et les méthodes de la classe parente dans une classe enfant, ici avec des animaux et leurs races.

```
class Animal {
  constructor(nom) {
    this.nom = nom;
  }

  sePresenter() {
    console.log(`Je suis ${this.nom}`);
  }
}

class Chien extends Animal {
  constructor(nom, race) {
    super(nom);  // 1. Appelle le constructeur de la classe Animal
    this.race = race;
  }

  sePresenter() {
    super.sePresenter();  // 2. Appelle la méthode sePresenter de la classe Animal
    console.log(`Je suis un ${this.race}`);
  }

  aboyer() {
    console.log("Woof woof !");
  }
}

const chien = new Chien("Rex", "Labrador");
chien.sePresenter();
// "Je suis Rex"
// "Je suis un Labrador"
```

Appel du constructeur parent

Dans `Chien`, `super(nom)` initialise la propriété `nom` via le constructeur de la classe `Animal`.

Extension d'une méthode parent

La méthode `sePresenter()` de `Chien` appelle d'abord `super.sePresenter()` pour exécuter le comportement du parent, puis ajoute sa propre spécificité.

POLYMORPHISME

Polymorphisme : Même méthode, comportements différents

Le polymorphisme est un concept clé de la POO qui permet à des objets de différentes classes de répondre à la même méthode de manière unique. En d'autres termes, une même instruction peut avoir des résultats différents selon l'objet sur lequel elle est appliquée, ce qui rend le code plus flexible et extensible.

EXEMPLE CONCRET

Polymorphisme avec des formes géométriques

```
class Forme {
  constructor(couleur) {
    this.couleur = couleur;
  }

  calculerAire() {
    // Méthode abstraite (à redéfinir dans les enfants)
    throw new Error("Cette méthode doit être implémentée dans les classes enfants");
  }

  decrire() {
    console.log(`Forme de couleur ${this.couleur}, aire : ${this.calculerAire()}`);
  }
}

class Rectangle extends Forme {
  constructor(couleur, largeur, hauteur) {
    super(couleur);
    this.largeur = largeur;
    this.hauteur = hauteur;
  }

  calculerAire() {
    return this.largeur * this.hauteur;
  }
}

class Cercle extends Forme {
  constructor(couleur, rayon) {
    super(couleur);
    this.rayon = rayon;
  }

  calculerAire() {
    return Math.PI * this.rayon ** 2;
  }
}

// Polymorphisme en action
const formes = [
  new Rectangle("rouge", 10, 5),
  new Cercle("bleu", 7),
  new Rectangle("vert", 8, 8)
];

// Même méthode, comportements différents !
formes.forEach((forme) => forme.decrire());
// "Forme de couleur rouge, aire : 50"
// "Forme de couleur bleu, aire : 153.93..."
// "Forme de couleur vert, aire : 64"
```

MÉTHODES STATIQUES

Méthodes statiques

Les méthodes statiques appartiennent à la classe elle-même, pas aux instances. On les utilise pour des utilitaires ou des compteurs.

Appartenance à la classe

Une méthode statique est liée à la classe et non à ses instances. Elle est appelée directement sur la classe elle-même (ex: `MaClasse.maMethodeStatique()`).

Indépendance des instances

Elles ne peuvent pas accéder aux propriétés ou méthodes d'une instance spécifique (le mot-clé `this` ne fait pas référence à une instance de classe).

Cas d'utilisation courants

Idéales pour les fonctions utilitaires (comme la validation), les fabriques (factory methods), ou pour gérer des propriétés partagées comme des compteurs d'instances.

EXEMPLE CONCRET

Exemple Concret : Méthodes Statiques

```
class Personnage {
  static #compteur = 0;  // Propriété statique privée pour compter les instances
  static #personnages = [];  // Liste statique privée pour stocker toutes les instances

  constructor(nom) {
    this.nom = nom;
    this.id = ++Personnage.#compteur; // Incrémente le compteur statique
    Personnage.#personnages.push(this); // Ajoute l'instance à la liste statique
  }

  // Méthode statique pour obtenir le nombre total de personnages
  static obtenirNombrePersonnages() {
    return Personnage.#compteur;
  }

  // Méthode statique pour obtenir la liste de tous les personnages
  static obtenirTousLesPersonnages() {
    return Personnage.#personnages;
  }

  // Méthode statique pour trouver un personnage par son nom
  static trouverParNom(nom) {
    return Personnage.#personnages.find((p) => p.nom === nom);
  }
}

// Utilisation des méthodes statiques (sans créer d'instance)
const p1 = new Personnage("Arthur");
const p2 = new Personnage("Merlin");
const p3 = new Personnage("Lancelot");

console.log(Personnage.obtenirNombrePersonnages());  // Affiche 3
console.log(Personnage.trouverParNom("Merlin"));     // Affiche { nom: "Merlin", id: 2 }

// Impossible sur une instance :
// p1.obtenirNombrePersonnages();  // Erreur !
```

MISE EN PRATIQUE

Construction d'une hiérarchie d'aventuriers

Créez une hiérarchie de classes en JavaScript pour différents types d'aventuriers, chacun doté de compétences et de propriétés uniques, en appliquant les principes de la POO.

1. **Classe de base** `Adventurer` Implémentez une classe de base `Adventurer` avec des propriétés privées (nom, classe, niveau) et un constructeur. Incluez des propriétés et méthodes statiques pour gérer toutes les instances (compteur, liste d'aventuriers, recherche par nom). Définissez une méthode publique abstraite `showStatistics()` que les classes enfants devront redéfinir.
2. **Classes spécialisées** Créez des classes dérivées de `Adventurer` pour différents archétypes, chacune avec ses propres statistiques, une propriété unique et des méthodes spécifiques. Voir slide suivante pour les spécificités.
3. **Polymorphisme** Assurez-vous que chaque classe spécialisée redéfinit la méthode `showStatistics()` de manière à inclure ses propriétés et spécificités uniques.

MISE EN PRATIQUE

Construction d'une hiérarchie d'aventuriers

Guerrier
Force élevée, intelligence faible.

Propriété unique : **rage** (0 à 100).

Méthodes : `rageAttack()`, `defensiveStance()`.

Mage
Intelligence élevée, force faible.

Propriété unique : **spells** (liste des sorts connus).

Méthodes : `castSpell()`, `learnSpell()`, `showSpells()`.

Voleur
Agilité élevée, force moyenne.

Propriété unique : **stealth** (chance de coup critique).

Méthodes : `sneakAttack()`, `steal()`, `hide()`.

Prêtre
Intelligence moyenne, force faible.

Propriété unique : **faith** (efficacité des soins).

Méthodes : `heal()`, `benediction()`, `resurrection()`.

Archer
Agilité élevée, force moyenne.

Propriété unique : **arrows** (munitions, 30 au départ).

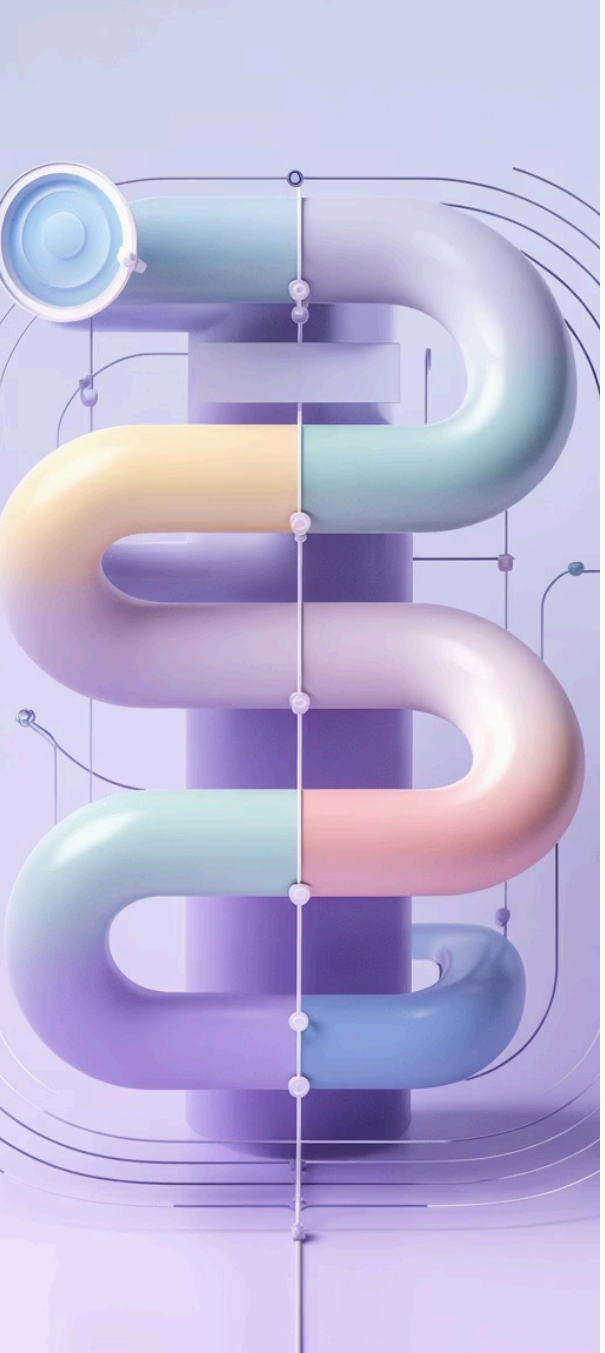
Méthodes : `precisionShot()`, `rainOfArrows()`, `craftArrows()`.

CONCLUSION

Et… on s'arrête là pour les notions du jour !

Ça fait beaucoup (vraiment), donc on fait une pause pour aujourd'hui.

… mais ce n'est pas tout à fait fini 😏



À SUIVRE

Prochaines étapes

Demain, on exploitera tout ce qu'on a pu voir aujourd'hui pour revenir sur un concept courant dans le développement : le design-pattern MVC.

Pas de back-end, MVC côté client (navigateur, donc front-end) !

Et comme d'habitude, ce qu'on a vu jusqu'à aujourd'hui sera approfondi au fur et à mesure !

PROJET DE FIN DE JOURNÉE

Arène de combat

Plongez dans l'univers de la programmation orientée objet en construisant une arène de combat complète où aventuriers et monstres s'affrontent dans des duels automatisés.

Vos aventuriers

Exploitez la hiérarchie d'aventuriers créée précédemment, chacun avec ses compétences uniques.

Des monstres

Créez une classe `Monster` pour des adversaires variés (faciles, moyens, difficiles).

Le cœur du combat

Implémentez la logique de combat tour par tour avec la classe `Battle`.

Gestion de l'arène

La classe `Arena` orchestrera tous les affrontements et suivra les statistiques.

Des interfaces conçues avec JavaScript seront obligatoires !